

The CAP Theorem

Understanding the five database genres is important when deciding on which database to use in a particular case, but there's one other major thing that you should always bear in mind. A recurring theme in this book has been the CAP theorem, which lays bare an unsettling truth about how distributed database systems behave in the face of network instability.

CAP proves that you can create a distributed database that can have one or more of the following qualities: It can be *consistent* (writes are atomic and all subsequent requests retrieve the new value), *available* (the database will always return a value as long as a single server is running), and/or *partition tolerant* (the system will still function even if server communication is temporarily lost—that is, a network partition). But the catch is that any given system can be *at most* two of these things at once, and *never* all three.

In other words, you can create a distributed database system that is *consistent* and *partition tolerant* (a “CP” system), a system that is *available* and *partition tolerant* (an “AP” system), or a system that is *consistent* and *available* (the much more rare CA system that is not *partition tolerant*—which basically means not distributed). Or a system can have only one of those qualities (this book doesn't cover any databases like that, and you're unlikely to encounter such a database in wide use). But at the end of the day it simply isn't possible to create a distributed database that is consistent and available and partition tolerant at the same time, and anyone who says that they've “solved CAP” is saying that they've defied the laws of physics and thus should not be trusted.

The CAP theorem is pertinent when considering a distributed database because it forces you to decide what you are willing to give up. The database you choose will lose either availability or consistency. Partition tolerance is strictly an architectural decision (depending on whether you want a distributed database). It's important to understand the CAP theorem to fully grok your options. The

trade-offs made by the database implementations in this book are largely influenced by it.

A CAP Adventure, Part I: CAP

Imagine the world as a giant distributed database system. All of the land in the world contains information about certain topics, and as long as you're *somewhere* near people or technology, you can find an answer to your questions.

Now, for the sake of argument, imagine you are a passionate Beyoncé fan and the date is September 5, 2016. Suddenly, while at your friend's beach house party celebrating the release of Beyoncé's hot new studio album, a freak tidal wave sweeps across the dock and drags you out to sea. You fashion a makeshift raft and wash up on a desert island days later. Without any means of communication, you are effectively *partitioned* from the rest of the system (the world). There you wait for five long years...

One morning in 2021, you are awakened by shouts from the sea. A salty old schooner captain has discovered you! After five years alone, the captain leans over the bow and bellows: "How many studio albums does Beyoncé have?"

You now have a decision to make. You can answer the question with the most recent value you have (which is now five years old). If you answer his query, you are *available*. Or, you can decline to answer the question, knowing that because you are partitioned, your answer may not be *consistent* with the current state of the world. The captain won't get his answer, but the state of the world remains consistent (if he sails back home, he can get the correct answer). In your role of queried node, you can either help keep the world's data *consistent* or be *available*, but *not both*.

Eventual Consistency

Distributed databases must be partition tolerant, so the choice between availability and consistency can be difficult. However, while CAP dictates that if you pick availability you cannot have true consistency, you can still provide *eventual consistency*.

The idea behind eventual consistency is that each node is always available to serve requests. As a trade-off, data modifications are propagated in the background to other nodes. This means that at any time the system may be inconsistent, but the data is still largely accurate.

The Internet's Domain Name Service (DNS) is a prime example of an eventually consistent system. You register a domain, and it may take a few days to propagate to all DNS servers across the Internet. But at no time is any particular DNS server unavailable (assuming you can connect to it, that is).

A CAP Adventure, Part II: Eventual Consistency

Let's rewind two years, back to 2009. You've been on the island for three years at this point, and you spot a bottle in the sand—precious contact with the outside world. You uncork it and rejoice! You've just received an integral piece of knowledge...

The number of studio albums Beyoncé has is of utmost importance to the world's aggregate knowledge. It's so important, in fact, that every time she releases a new album, someone writes the current date and the number on a piece of paper. They place that paper in a bottle and throw it out to sea. If someone, like yourself, is partitioned from the rest of the world on a desert island, they can *eventually* have the correct answer.

Skip forward to the present. When the ship captain asks, "How many studio albums does Beyoncé have?" you remain *available* and answer "three." You may be *inconsistent* with the rest of the world, but you are reasonably certain of your answer, having not yet received another bottle.

The story ends with the captain rescuing you, and you return home to find her new album and live happily ever after. As long as you remain on land, you needn't be partition tolerant and can remain consistent and available until the end of your days.

CAP in the Wild

The databases in this book largely occupy one corner or another of the CAP trade-off triangle. Redis, PostgreSQL, and Neo4J are consistent and available (CA); they don't distribute data and so partitioning is not an issue (though arguably, CAP doesn't make much sense in non-distributed systems). MongoDB and HBase are generally consistent and partition tolerant (CP). In the event of a network partition, they can become unable to respond to certain types of queries (for example, in a Mongo replica set you flag slaveok to false for reads). In practice, hardware failure is handled gracefully—other still-networked nodes can cover for the downed server—but strictly speaking, in the CAP theorem sense, they are unavailable. Finally, CouchDB is available and partition tolerant (AP). Even though two or more CouchDB servers can replicate data between them, CouchDB doesn't guarantee consistency between any two servers.

It's worth noting that most of these databases can be configured to change CAP type (Mongo can be CA, CouchDB can be CP), but here we've noted their default or common behaviors.

The Latency Trade-Off

There is more to distributed database system design than CAP, however. For example, low latency (speed) is a chief concern for many architects. If you read

the Amazon Dynamo¹ paper, you'll notice a lot of talk about availability but also Amazon's latency requirements. For a certain class of applications, even a small latency change can translate to a large costs. Yahoo's PNUTS database famously gives up both availability on normal operation and consistency on partitions in order to squeeze a lower latency out of its design.² It's important to consider CAP when dealing with distributed databases, but it's equally important to be aware that distributed database theory does not stop there.

-
1. <http://allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
 2. <http://dbmsmusings.blogspot.com/2010/04/problems-with-cap-and-yahoos-little.html>